

ABASYN

UNIVERSITY

Assignment : 01

Submitted By:

Alina Mubarik

9067

TO: MA'AM SADAF

SUBJECT: MOBILE APPLICATION DEVELOPMENT (CS-375)

BATCH: SPRING-2024

DEPARTMENT OF COMPUTER SCIENCE

**ABASYN UNIVERSITY
ISLAMABAD CAMPUS**

Software Requirements Specification (SRS)

DineXpert – Restaurant Mobile Application

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements of the **DineXpert Restaurant Application**, a mobile-based system developed using React Native.

The purpose of this application is to provide users with a seamless digital restaurant experience, including browsing menus, placing orders, managing carts, making reservations, and viewing order history.

1.2 Scope

The following functionalities are included in the initial version (MVP):

- User login (basic authentication)
 - Dashboard overview
 - Menu browsing with images
 - Add to cart functionality
 - Order placement system
 - Order history tracking
 - Table reservation system with QR code
 - Logout functionality
 - Bottom navigation system
-

1.3 Definitions and Acronyms

Term	Description
MVP	Minimum Viable Product

Term	Description
UI	User Interface
UX	User Experience
QR Code	Quick Response Code used for reservation verification
React Native Framework	used to build mobile applications
NoSQL	Non-relational database structure

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Browse menu, add items to cart, place orders, view orders, make reservations, logout

2.2 User Stories

- As a user, I want to log in so that I can access the application
 - As a user, I want to view the menu so that I can choose food items
 - As a user, I want to add items to the cart so that I can place an order
 - As a user, I want to place an order so that I can receive food
 - As a user, I want to see my previous orders
 - As a user, I want to reserve a table and get a QR code
 - As a user, I want to logout securely
-

3. Functional Requirements

- The system shall allow users to log in using their name
- The system shall display a dashboard after login
- The system shall display menu items with images and prices
- The system shall allow users to add items to the cart
- The system shall update quantity if item already exists in cart
- The system shall calculate total cart price

- The system shall allow users to place orders
 - The system shall generate unique order IDs
 - The system shall store and display order history
 - The system shall allow users to reserve a table
 - The system shall generate a QR code for reservation
 - The system shall allow users to logout
-

4. Non-Functional Requirements

- The application shall have a responsive and user-friendly UI
 - The system shall be fast and lightweight (optimized for mobile)
 - The application shall maintain data consistency during session
 - The UI shall follow a consistent purple theme design
 - The system shall provide quick feedback (alerts/messages)
 - The application shall be compatible with Android and iOS (via Expo)
-

5. Database Schema (NoSQL)

Collection Name	Fields	Description
Users	userId, name	Stores user information
Menu	id, name, price, image	Stores food items
Cart	itemId, userId, quantity	Stores cart items
Orders	orderId, userId, items, total, status	Stores placed orders
Reservations	reservationId, userId, qrData	Stores reservation details

6. MVP Frontend Development Overview (React Native)

```
import 'react-native-gesture-handler';

import React, { useState } from "react";

import {
```

```

View,
Text,
TextInput,
TouchableOpacity,
FlatList,
Image,
StyleSheet,
Alert,
} from "react-native";

import QRCode from "react-native-qrcode-svg";
import { NavigationContainer } from "@react-navigation/native";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";

```

```

const Tab = createBottomTabNavigator();

```

```

/* ----- MENU DATA ----- */

```

```

const MENU = [
  {
    id: 1,
    name: "Zinger Burger",
    price: 6,
    image: "https://images.unsplash.com/photo-1550547660-d9450f859349",
  },
  {
    id: 2,
    name: "Cheese Pizza",
    price: 9,
    image: "https://images.unsplash.com/photo-1548365328-9f547f6d0e1b",
  },
  {
    id: 3,
    name: "Cream Pasta",
    price: 7,
    image: "https://images.unsplash.com/photo-1525755662778-989d0524087e",
  },
];

```

```

/* ----- LOGIN ----- */

function Login({ setUser }) {

  const [name, setName] = useState("");

  return (

    <View style={styles.login}>

      <Text style={styles.logo}> DineXpert</Text>

      <Text style={styles.subtitle}>Restaurant Dashboard System</Text>

      <TextInput

        placeholder="Enter Name"

        placeholderTextColor="#ccc"

        style={styles.input}

        onChangeText={setName}

      />

      <TouchableOpacity

        style={styles.primaryBtn}

        onPress={() => {

          if (!name) return Alert.alert("Error", "Enter name first");

          setUser(name);

        }}

      >

        <Text style={styles.btnText}>Login</Text>

      </TouchableOpacity>

    </View>

  );
}

```

```

/* ----- DASHBOARD ----- */

function Dashboard({ user, setUser }) {

  return (

    <View style={styles.container}>

      <Text style={styles.title}>Welcome, {user} </Text>

      <View style={styles.card}>

        <Text style={{ fontSize: 16 }}> Browse Menu</Text>

      </View>

    </View>

  );
}

```

```
</View>
```

```
<View style={styles.card}>  
  <Text style={{ fontSize: 16 }}> Track Orders</Text>  
</View>
```

```
<View style={styles.card}>  
  <Text style={{ fontSize: 16 }}> Make Reservation</Text>  
</View>
```

```
<TouchableOpacity  
  style={[styles.primaryBtn, { marginTop: 20 }]}  
  onPress={() => setUser(null)}  
>  
  <Text style={styles.btnText}>Logout</Text>  
</TouchableOpacity>  
</View>  
);  
}
```

```
/* ----- MENU ----- */
```

```
function Menu({ cart, setCart }) {  
  const addToCart = (item) => {  
    const exist = cart.find((c) => c.id === item.id);
```

```
    if (exist) {  
      setCart(  
        cart.map((c) =>  
          c.id === item.id ? { ...c, qty: c.qty + 1 } : c  
        )  
      );  
    } else {  
      setCart([...cart, { ...item, qty: 1 }]);  
    }  
  }  
}
```

```
Alert.alert("Added", item.name);  
};
```

```

return (
  <View style={styles.container}>
    <Text style={styles.title}>Menu</Text>

    <FlatList
      data={MENU}
      keyExtractor={({i}) => i.id.toString()}
      renderItem={({ item }) => (
        <View style={styles.card}>
          <Image source={{ uri: item.image }} style={styles.image} />
          <Text style={styles.food}>{item.name}</Text>
          <Text style={styles.price}>${item.price}</Text>

          <TouchableOpacity
            style={styles.secondaryBtn}
            onPress={() => addToCart(item)}
          >
            <Text style={styles.btnText}>Add to Cart</Text>
          </TouchableOpacity>
        </View>
      )}
    />
  </View>
);
}

```

```

/* ----- CART + ORDER ----- */
function Cart({ cart, setCart, orders, setOrders }) {
  const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0);

```

```

  const placeOrder = () => {
    if (cart.length === 0)
      return Alert.alert("Empty Cart", "Add items first");

```

```

    const newOrder = {
      id: Date.now(),

```

```

    total,
    status: "Confirmed ✔",
  });

```

```

setOrders([newOrder, ...orders]);
setCart([]);

```

```

Alert.alert(" Order Placed!", `Order ID: ${newOrder.id}`);
};

```

```

return (
  <View style={styles.container}>
    <Text style={styles.title}>Cart</Text>

```

```

    {cart.map((i) => (
      <Text key={i.id}>
        {i.name} x {i.qty}
      </Text>
    ))}

```

```

    <Text style={styles.total}>Total: ${total}</Text>

```

```

    <TouchableOpacity style={styles.primaryBtn} onPress={placeOrder}>
      <Text style={styles.btnText}>Place Order</Text>
    </TouchableOpacity>
  </View>
);
}

```

```

/* ----- ORDERS ----- */

```

```

function Orders({ orders }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Orders</Text>

```

```

      {orders.length === 0 ? (

```

```

    <Text>No orders yet</Text>

    ) : (
      orders.map((o) => (
        <View key={o.id} style={styles.card}>
          <Text>Order ID: {o.id}</Text>
          <Text style={{ color: "#6a0dad" }}>{o.status}</Text>
          <Text>Total: ${o.total}</Text>
        </View>
      ))
    )}
  </View>
);
}

```

```

/* ----- RESERVATION ----- */
function Reservation({ user }) {
  return (
    <View style={styles.center}>
      <Text style={styles.title}>Reservation</Text>

```

```

      <Text>Reserved for {user}</Text>

```

```

      <QRCode value={`DineXpert-${user}`} size={200} />
    </View>
  );
}

```

```

/* ----- MAIN APP ----- */
function MainApp({ user, setUser }) {
  const [cart, setCart] = useState([]);
  const [orders, setOrders] = useState([]);

```

```

  return (
    <Tab.Navigator
      screenOptions={{
        headerStyle: { backgroundColor: "#6a0dad" },
        headerTintColor: "#fff",

```

```

    tabBarStyle: { backgroundColor: "#6a0dad" },
    tabBarActiveTintColor: "#fff",
  }}
>
<Tab.Screen name="Dashboard">
  {() => <Dashboard user={user} setUser={setUser} />}
</Tab.Screen>

```

```

<Tab.Screen name="Menu">
  {() => <Menu cart={cart} setCart={setCart} />}
</Tab.Screen>

```

```

<Tab.Screen name="Cart">
  {() => (
    <Cart
      cart={cart}
      setCart={setCart}
      orders={orders}
      setOrders={setOrders}
    />
  )}
</Tab.Screen>

```

```

<Tab.Screen name="Orders">
  {() => <Orders orders={orders} />}
</Tab.Screen>

```

```

<Tab.Screen name="Reserve">
  {() => <Reservation user={user} />}
</Tab.Screen>
</Tab.Navigator>
);
}

```

```

/* ----- ROOT ----- */
export default function App() {
  const [user, setUser] = useState(null);

```

```

if (!user) return <Login setUser={setUser} />;

return (
  <NavigationContainer>
    <MainApp user={user} setUser={setUser} />
  </NavigationContainer>
);
}

/* ----- STYLES ----- */
const styles = StyleSheet.create({
  container: { flex: 1, padding: 15, backgroundColor: "#f8f5ff" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },

  login: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "#4B0082",
  },

  logo: { fontSize: 30, color: "fff", fontWeight: "bold" },
  subtitle: { color: "ddd", marginBottom: 20 },

  input: {
    width: "80%",
    borderWidth: 1,
    borderColor: "fff",
    color: "fff",
    padding: 10,
    borderRadius: 10,
    marginBottom: 10,
  },

  title: { fontSize: 22, fontWeight: "bold", marginBottom: 10 },

```

```
primaryBtn: {  
  backgroundColor: "#9b59b6",  
  padding: 12,  
  borderRadius: 10,  
  marginTop: 10,  
},
```

```
secondaryBtn: {  
  backgroundColor: "#6a0dad",  
  padding: 8,  
  borderRadius: 8,  
  marginTop: 10,  
},
```

```
btnText: { color: "white", textAlign: "center", fontWeight: "bold" },
```

```
card: {  
  backgroundColor: "white",  
  padding: 15,  
  borderRadius: 15,  
  marginBottom: 10,  
},
```

```
image: { width: "100%", height: 150, borderRadius: 10 },
```

```
food: { fontSize: 18, fontWeight: "bold", marginTop: 5 },
```

```
price: { color: "#6a0dad", fontWeight: "bold" },
```

```
total: { fontSize: 18, fontWeight: "bold", marginTop: 10 },  
});
```

6.1 Login Screen

- User enters name
- Validates input
- Navigates to dashboard

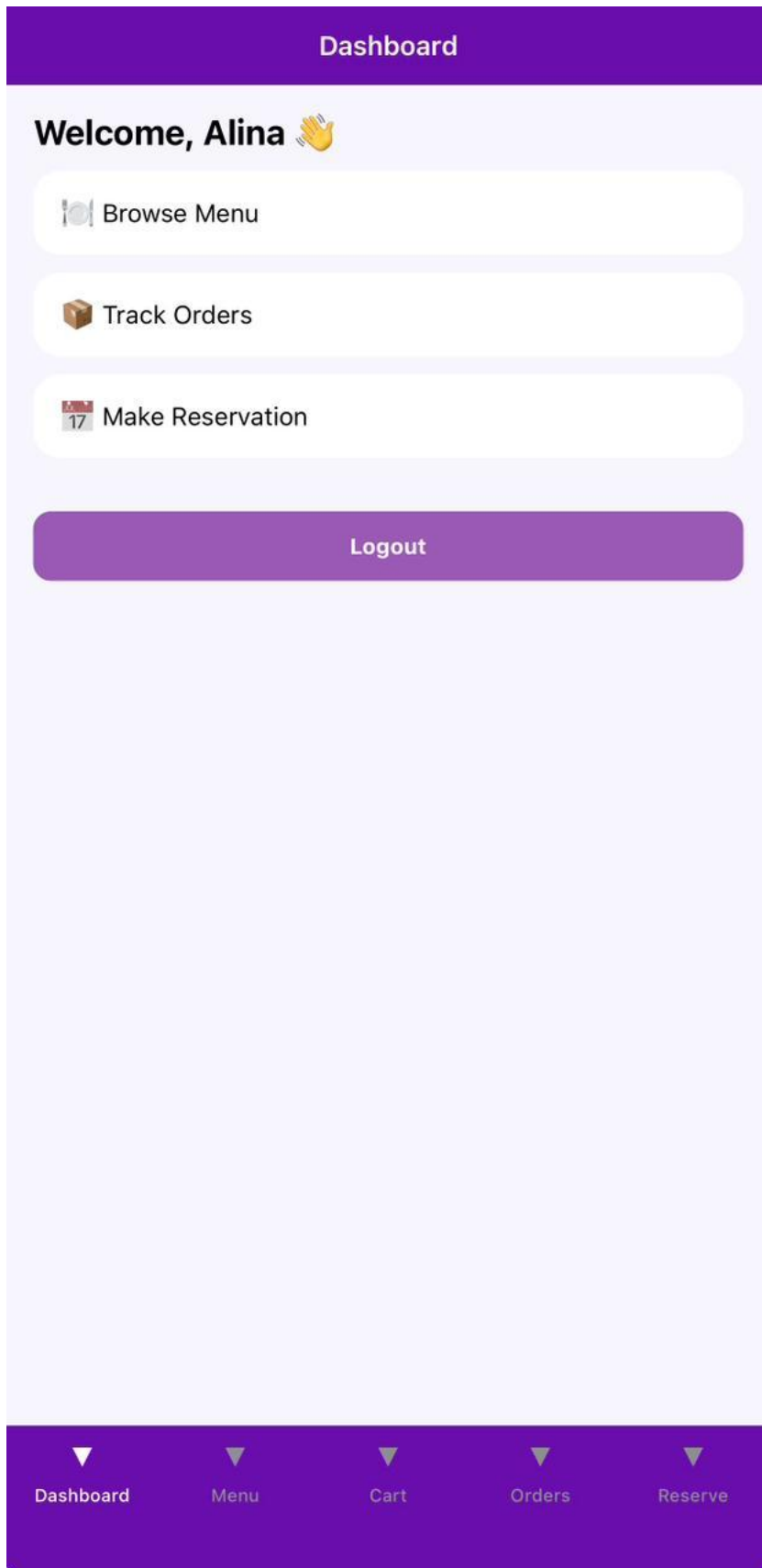
 DineXpert
Restaurant Dashboard System

Enter Name

Login

6.2 Dashboard

- Displays welcome message
- Navigation to features
- Logout button



6.3 Menu Screen

- Displays food items with images
- Add to cart button

Menu

Menu



Zinger Burger

\$6

Add to Cart

Cheese Pizza

\$9

Add to Cart



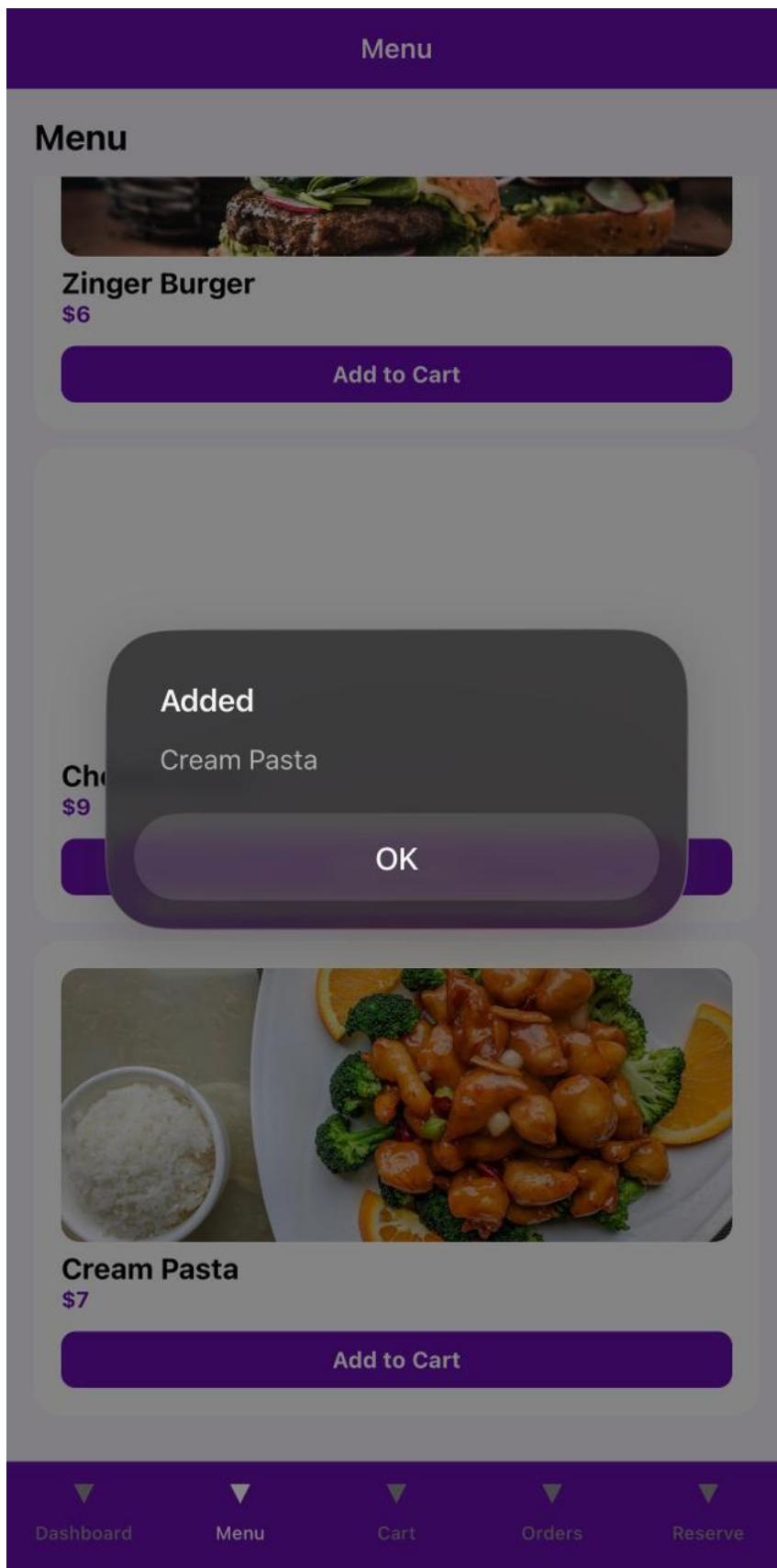
▼
Dashboard

▼
Menu

▼
Cart

▼
Orders

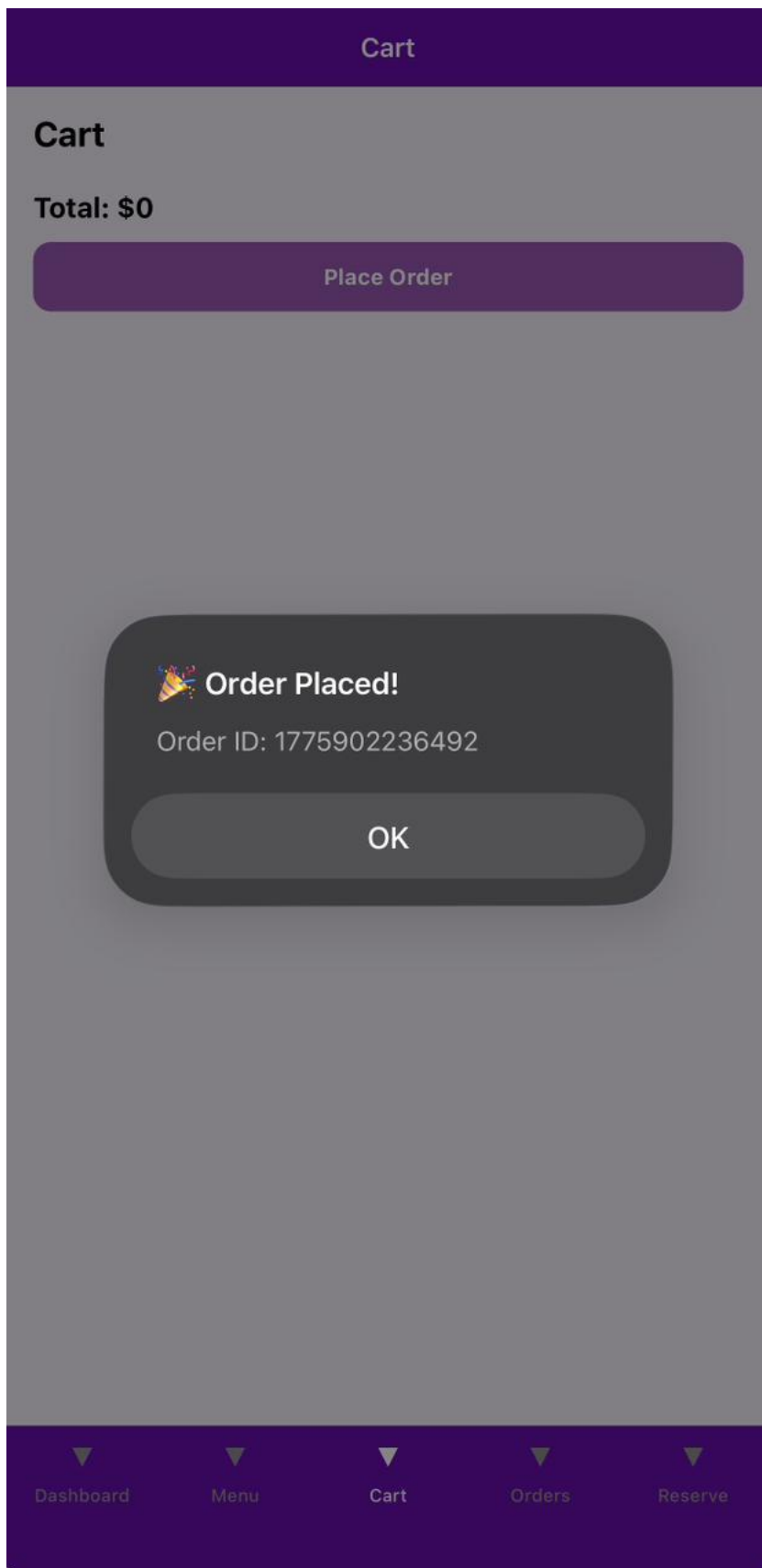
▼
Reserve



6.4 Cart Screen

- Shows selected items
- Displays total price
- Place order button





6.5 Orders Screen

- Displays order history
- Shows order ID, status, total

Orders

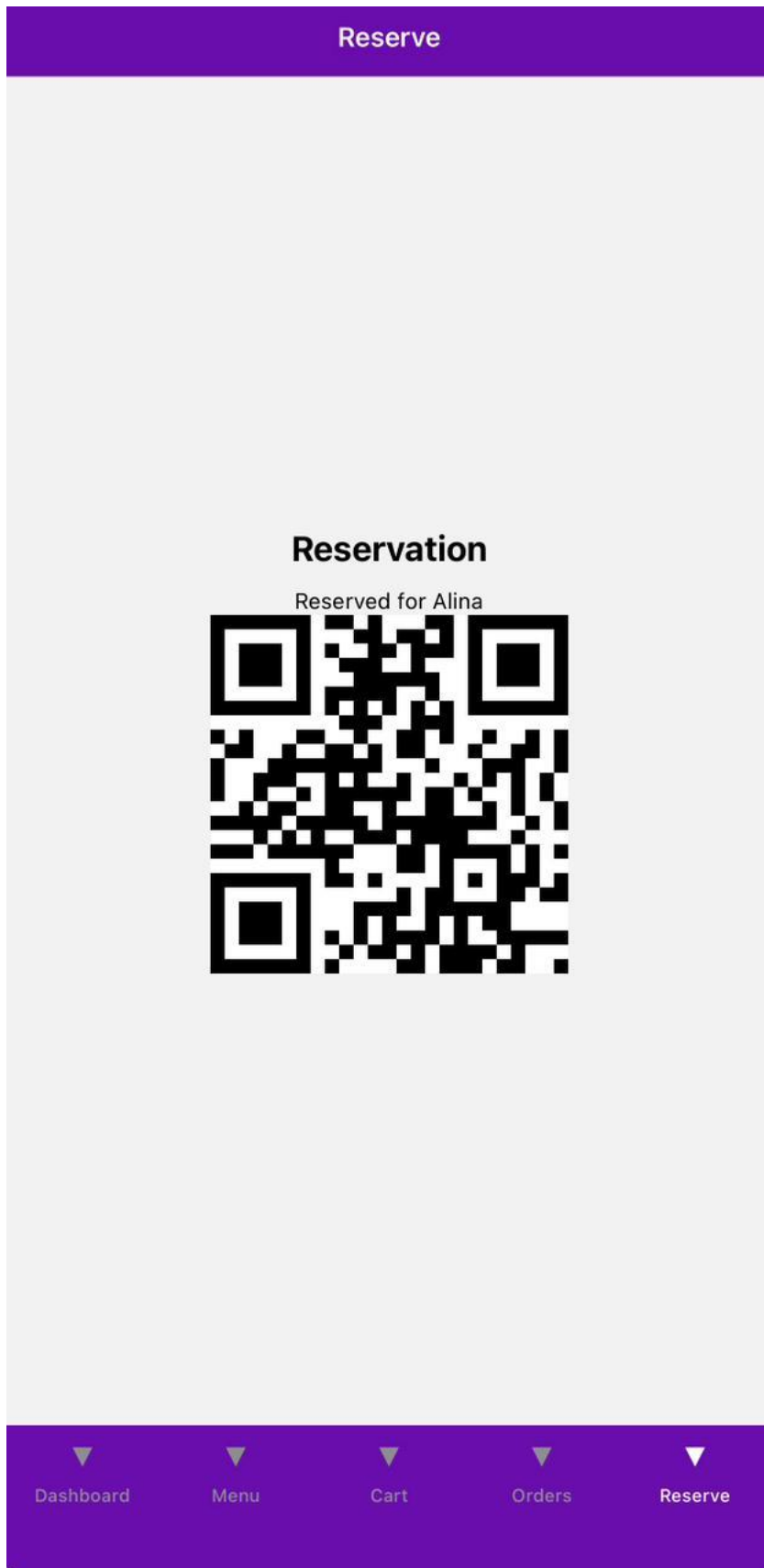
Orders

Order ID: 1775902236492
Confirmed ✓
Total: \$13

- ▼
Dashboard
- ▼
Menu
- ▼
Cart
- ▼
Orders
- ▼
Reserve

6.6 Reservation Screen

- Displays reservation confirmation
- Generates QR code



7. Conclusion

The **DineXpert Restaurant Application** successfully demonstrates a modern, user-friendly mobile solution for restaurant management and customer interaction. The system integrates essential features such as menu browsing, cart management, order placement, reservation with QR code generation, and user authentication within a clean and visually appealing interface.

The application is designed following a modular and scalable architecture, making it suitable for future enhancements such as real-time order tracking, payment gateway integration, and backend database connectivity.

Overall, this project fulfills the objectives of developing a functional Minimum Viable Product (MVP) while maintaining professional UI/UX standards and efficient system performance. It provides a strong foundation for a full-scale restaurant management system and reflects practical implementation of software engineering principles.